

HOLY ANGEL UNIVERSITY**FINAL PROJECT PROPOSAL REVISED**

In Partial Fulfillment of the Requirements for
Applications Development and Emerging Technologies

**CS-301**

Matthew Simon R. Green

Submitted to:
Prof. Tjakoen A. Stolk

September 20, 2026

HOLY ANGEL UNIVERSITY**App Name: PokéBinder****The Problem, in One Sentence**

People shopping at card shops or browsing online listings may not remember if they already own a card, what condition it is in, or which binder it is stored in, which can lead to duplicate purchases or time spent searching through physical binders.

Who Is This For?

This app is for me and other Pokémon TCG collectors who keep their cards in physical binders but do not have one place to track their collection. Collectors often rely on memory or check their binders manually. When they are away from home, they may have to guess or skip a purchase because they are unsure whether they already own the card.

Core Features (MVP)

Since I have already started coding PokéBinder, I am already ahead in the development process. I have built the screens and main user flows for all of my planned MVP features except for real card scanning. The scanner screen and its overall flow are already built, but I have not yet fully implemented or tested actual card recognition.

None of the MVP features saves data yet, so my main remaining work is connecting the existing screens to Supabase rather than building new UI from scratch. Several parts of PokéBinder still use sample or in-memory data. I plan to connect these parts to Supabase so that the data can be properly stored, retrieved, and updated even after the app is closed.

This means most of my remaining work will focus on connecting the existing features to real data and making sure they work properly together. It also gives me more time to improve the functionality of the features I have already built instead of creating the main UI from scratch.

HOLY ANGEL UNIVERSITY

Feature	Still in the MVP?	Flutter Pieces It Needs	Honest Estimate
Sign in / Sign up	Yes	Form, TextField, supabase_flutter auth, auth-state listener, Navigator.pushAndRemoveUntil	4 hours
Manage Collection	Yes	Form (CardFormScreen), ListView.separated, GridView for binder pages, showDialog for delete confirmation, model classes with toJson/fromJson	8 hours
Search, Browse & Card Details	Yes	SearchBar, TabBar, ListView.separated, GridView, GestureDetector + Transform for 3D card tilt, Navigator.push	3 hours
Card Scanner	Yes, demo flow only	CustomPaint scan overlay, AnimationController for the existing animation; real version would need camera and text recognition, which have not been tested yet	12+ hours / Risk
Wishlist, Trade List & Deck Planner	Yes	TabBar, Form, showDialog, showModalBottomSheet, Dismissible, deck-card join	9 hours
Trainer Card & Collection Overview	Yes	Form, CircleAvatar, CustomPaint for statistics, favorite pickers	3 hours
Supabase Setup	New	Dashboard setup, --dart-define, GitHub Actions secrets	6 hours

My current firm estimate is **33 hours**, not including real scanner recognition because that feature has been moved to a stretch goal. I have 3 weeks remaining for the project, which gives me enough time to complete the MVP and make final improvements.

The Card Scanner is listed separately as a risk because I have already built the UI and animation, but I have not yet fully tested the actual camera and text recognition implementation. The 12+ hour estimate is therefore not a firm estimate.

HOLY ANGEL UNIVERSITY

Stretch Goals

Future features could include:

1. **Real Scanner Recognition** – Use the camera and on-device text recognition to identify a Pokémon card by reading its name and card number.
2. **Real Card Catalog** – Connect PokéBinder to the Pokémon TCG API so users can search for any card instead of being limited to cards I have already entered into the app.
3. **Live Prices** – Connect to a card pricing source so the estimated value is updated with current market prices instead of using hard-coded values for each card.
4. **Set Completion Tracking** – Track progress and show which cards are still missing from a set.
5. **Deck Sharing & Friends List** – Allow users to share their decks and connect with other collectors through a friends list.
6. **AI Assistant** – Help users identify cards, create and organize binders, plan decks, and answer questions about the app.

How My App Saves Data

Will Different Users See the Same Data?

No. Each person who uses PokéBinder should have their own private collection. If two people install the app, they should not see each other's cards, binders, decks, wishlist, trade list, or trainer card. This is important to me because PokéBinder is meant to represent a person's actual physical collection. The cards and information I save should belong only to that user's account.

How Much Data Will the App Store?

The app currently has 20 sample cards, but a real user could have around 300 to 1,500 cards in their collection. A user might also add or edit around 20 to 100 cards in a typical week. Aside from cards, a user could have around 5 to 15 binders, about 30 wishlist and trade entries, and several decks.

HOLY ANGEL UNIVERSITY

Based on this, I estimate that PokéBinder could store around **500 to 2,000 records per user**.

My Choice: Supabase

I chose **Supabase** because it provides PostgreSQL, Supabase Auth, and Row Level Security in one platform. I will connect it to Flutter using `supabase_flutter`.

One reason I chose Supabase is that I have already built the Sign In, Sign Up, and Forgot Password screens. At the moment, these screens are mostly UI and do not have real account functionality behind them. Supabase Auth will allow me to connect these existing screens to actual user accounts.

Another reason is that my data is relational. For example, a deck can contain multiple cards, a card can belong to a specific binder and page, and a wishlist entry can be connected to a card that I already own. PostgreSQL works well for this because I can use separate tables and relationships between them.

I also want PokéBinder to work across devices. One of the main reasons for creating the app is to be able to check my collection while I am at a card shop and manage it again when I get home. Cloud storage allows the same account and collection to be accessed from different devices.

The Tradeoff

I know that using Supabase also adds more work to the project. A local database would be simpler to set up and could work without an internet connection.

However, I decided that cloud storage fits the purpose of PokéBinder better. I accept the additional setup required for authentication, Row Level Security, network access, and keeping sensitive keys out of my public repository.

If the connection is lost, the app will keep showing the last loaded data. Changes made while offline will not be saved, and the app will show an error message so the user can try again when connected.

What I Will Save

I plan to use five main tables and one join table:

- **profiles** – stores each user's trainer information
- **binders** – stores the user's binders
- **cards** – stores the cards in the user's collection
- **decks** – stores the user's decks
- **deck_cards** – connects cards to decks and stores their quantities
- **wishlist_entries** – stores wishlist and trade entries

The card artwork will remain bundled with the application for now. Instead of storing the actual image in the database, the database will store a reference to the image.

Have I Tested Supabase Yet?

As mentioned earlier, I have only used sample data that is currently hard-coded or stored in memory. I have not yet connected PokéBinder to Supabase or tested saving and retrieving real data from the database.

My first Supabase test will be a small one-hour spike. I will create a cards table, add a basic Row Level Security policy, and test inserting and reading one row from Flutter. I plan to complete this by **September 23, 2026**.

After that, I can use what I learn from the test to start moving the existing collection data into Supabase.

One Thing I Want to Add That the Course Did Not Teach

One feature I would like to add is a real Pokémon TCG card catalog using the **Pokémon TCG API** and the http package.

Right now, PokéBinder only has bundled sample cards, but I plan to add multiple complete sets in the future. With the Pokémon TCG API, I could search for real Pokémon cards instead of being limited to the cards already in the app. This would make adding cards to my collection more useful because I would not have to manually enter the information for every card.

HOLY ANGEL UNIVERSITY

The API could also provide real card information and pricing data, which could eventually support the Live Prices feature.

Does It Run Where I Develop?

The API-based card catalog should work on the platforms where my Flutter app currently runs, including the web. However, I still need to verify the API's current terms and rate limits before depending on it for the final version.

The real scanner is different. Actual card recognition would require camera access and text recognition or machine learning tools, which I have not fully implemented or tested yet.

Web Fallback and Demo Plan

The scanner already works as a browser demo. It shows the scanning animation and cycles through sample cards instead of using a real camera.

I plan to clearly label this as **Demo Mode** in the application so users know that it is simulated. If I successfully implement the real scanner, I plan to demonstrate that version using a phone for my final video.

For the web version, the scanner can continue using Demo Mode so that the rest of the application can still be tested without requiring camera hardware.

Core or Stretch Goal?

The real card catalog and real scanner are **stretch goals**.

PokéBinder can still perform its main purpose without them because users can manually add cards and use the sample card library. My priority is to make the existing collection system work with persistent data first. Once that is working properly, I can start working on these additional features.

HOLY ANGEL UNIVERSITY

How My Project Runs When Someone Else Opens It

Am I Keeping the DevicePreview Wrapper?

Yes. I am keeping the DevicePreview wrapper so the web version can be presented in a phone-sized layout. It is currently disabled during normal development, and I will set `enabled: true` in `main.dart` before my final deployment.

Can I Run the Project with `flutter run -d web-server`?

Yes, for the current sample-data version. The app can be run using the web server while the scanner is in Demo Mode. Before the final version, I still need to verify that the Supabase connection and environment variables work correctly in the web build.

If the required Supabase keys are missing, the app will show a setup message or use sample data instead of crashing or displaying a blank screen.

Does Anything That Needs Real Hardware Fall Back to Sample Data Instead of Crashing?

Yes. The only hardware feature is the Card Scanner, and it currently runs in Demo Mode. It shows a scanning animation and cycles through sample cards without using a camera, so it works in any browser.

If I implement real scanning later, the app will fall back to Demo Mode when the camera is unavailable, such as on the web or when camera permission is denied, instead of crashing.

Repository and Security

My project will be stored in my public GitHub repository: **`glorysonggreen/PokeBinder`**

My Supabase URL and publishable key will be kept in a local `.env` file that is ignored by Git. For deployment, I plan to use repository secrets instead of placing sensitive information directly in the source code.

The Supabase `service_role` key and any future Gemini API key will never be included in the Flutter application or public repository.

For testing, I will use public sample card information and a test account with a throwaway email. I will not store personal information in the sample data.

HOLY ANGEL UNIVERSITY

My .env.example file will only contain placeholder values so that someone setting up the project knows which variables are required without exposing my actual keys.

Data the App Remembers

Every field below comes from my model classes. Changes from the prelim are noted in the change log

Thing	Fields	Where it is saved
Trainer Card	user_id, name, title, bio, favorite_card_id, favorite_binder_id, favorite_deck_id	Supabase table profiles
Binder	id, user_id, name, description, page_count, slots_per_page, category, is_pinned, created_at	Supabase table binders
Card (owned)	id, user_id, binder_id, page, name, set_name, card_number, rarity, type, supertype, subtype, quantity_owned, condition, estimated_value, notes, image_ref, date_added	Supabase table cards
Deck	id, user_id, name, format, target_size, description, is_pinned, created_at	Supabase table decks
Deck card	deck_id, card_id, quantity	Supabase table deck_cards
Wishlist / Trade entry	id, user_id, name, set_name, card_number, rarity, condition, quantity, notes, kind (wishlist or trade), priority, estimated_value, asking_for, date_added, source_card_id, image_ref	Supabase table wishlist_entries

Screens

- Sign In, Sign Up, Forgot Password**
- Home Dashboard:** Collection Summary, Estimated Collection Value, Recent Cards, Quick Actions, Collection Statistics
- Collection & Binder:** Collection tab, Binder List tab, Binder Detail, Create/Edit Binder, Add Card to Binder, Card Details, Add/Edit Card
- Card Scanner (Demo Mode):** Scanner, then Add/Confirm Card

HOLY ANGEL UNIVERSITY

5. **Deck Planner:** Deck List, Deck Detail, Create/Edit Deck, Add Card to Deck
6. **More Hub:** Trainer Card, Edit Trainer Card, Choose Favorite Card, Collection & Statistics, Wishlist & Trade List, Wishlist Form, Trade Entry Form, Add Card to Trade List

Risks, Revised

The Risk I Named Last Time

The main risk I identified before was reliable card search and scanning. Search now works for my own collection, but it cannot find cards I have not entered yet, and scanning is still unresolved.

Search: Card search works, but it currently only searches cards that I have already entered. I still need to connect it to the Pokémon TCG API so that users can search for real cards instead of being limited to cards already entered into the collection.

Scanning: I have built the scanner flow around a simulation, so the main risk of recognizing a real Pokémon card has not been tested yet. The fallback of manually entering a card is already built, which helps reduce the impact if the scanner does not work as planned.

First step: Build a simple test call to the Pokémon TCG API that searches for a card by name and prints the results. I will do this by **September 23, 2026**. Real scanner development will start only after the MVP data has been successfully persisted.

A New Risk I Did Not See Before

A new risk I did not identify before is the persistence refactor. `PokemonCardData.library` is read or edited directly in 12 different screens. Moving from in-memory data to Supabase means that many of these operations will become asynchronous, which could break screens that are currently working.

First step: I will create one `CollectionRepository` class that uses the existing in-memory lists first. I can then move the screens to use the repository while keeping the current app working. Once that is stable, I can replace the in-memory data with Supabase. I will do this by **September 26, 2026**.

HOLY ANGEL UNIVERSITY

Another supporting risk is Row Level Security (RLS). If the policies are set up incorrectly, they could either prevent users from saving their own data or allow users to access someone else's data.

First step: I will test the RLS policies using two separate accounts by **September 29, 2026**, to make sure each account can only access its own data.

What Changed, and Why?

Section	Prelim Said	Now Says	Why It Changed
Core features	5 features, all in the MVP, nothing cut	Still 5, plus authentication. A feature is considered "done" only when it can save data. Real scanner recognition was moved to a stretch goal.	I built all the main screens, but scanner_screen.dart currently uses a 950 ms delay to cycle through 7 demo cards. I have not tested a camera plugin yet.
Auth	Not mentioned	Sign In, Sign Up, and Forgot Password added	I built these screens, but login currently accepts any input. They still need to be connected to real user accounts.
Storage	"Data the app needs to remember" with no storage chosen	Supabase using Postgres, Auth, and Row Level Security	Most of the collection currently comes from PokemonCardData.library and other static lists. The authentication screens also need a real backend.
Data	Card had a "binder name" text field; Deck was "selected cards"	binder_id foreign key, deck_cards join table, and fields such as rarity, priority, and estimated value added	My actual models (pokemon_card_data.dart, deck_data.dart, and wishlist_entry.dart) needed more information than I originally listed. Binders and deck contents also need their own related rows.

HOLY ANGEL UNIVERSITY

Screens	5 main screens with a few subscreens	6 groups, including authentication, Deck Detail, Add Card to Deck, and Edit Trainer Card	This now matches what is actually implemented in lib/screens/.
Stretch goals	3 items	6 items, including real scanner recognition, a real card catalog, and live prices	Card values are currently hard coded, and the app can only search for cards that I have already entered into the collection. These features would make PokéBinder more useful, but they are not required for the MVP.
Risks	Search and scanning reliability	Scanning remains unresolved, along with the persistence refactor. Each risk now has a first step and target date.	The persistence risk only became clear after I saw how many parts of the app directly read and modify the static lists.